

Adding to the Project

Intro.....	1
Updating UI.....	1
Overview.....	1
Normal Structure:.....	2
Adding Functionality (Services).....	2
Phrase Tracking.....	3
Implementation:.....	3
Adding Phrase Functionality:.....	3
TODOs:.....	3
Authentication.....	3
Adding New Roles.....	4
Statistics.....	4
Adding New Graphs on the Stats Page:.....	4
TODOs:.....	4
Cloud Functions.....	5
JavaScript Functions:.....	5
Python Functions:.....	5
TODOs:.....	5
Data.....	6
Database Structure:.....	6
Database Repository and Services:.....	6
TODOs:.....	6
Word Enrichment / WordBank.....	7
Implementation:.....	7
How It Works:.....	7
Adding Word Enrichment Features:.....	7
TODOs:.....	7
Localization:.....	8
TODOs:.....	8
Media (Videos and Images).....	8
Implementation:.....	8
How It Works:.....	9
Supported Formats:.....	9

Intro

This document contains short descriptions of how to add to each section of the project, written by the creators of those sections. Also, a short list of goals for each section. See the backlog for a more detailed list of Todo items.

Note on App Name: The application previously referred to as "Baby Words Tracker" is now named "WordBuds".

Updating UI

Overview

The pages for UI are housed in the pages folder. To add a new page simply add a new file and follow the format already existing. YouTube is an incredible resource for understanding different dart elements. There are so many short, effective dart tutorials there. **UI should be self-contained**. If you need storage between widgets or complex logic, you should create a service. See the **Adding Functionality (Services)** section for details.

Normal Structure:

Each of the existing pages follow a basic structure to ensure that the app theme matches throughout. This is done by using a scaffold that implements both the top bar and navigation bar. The exact structure can be seen below. The body section is where the page specific widgets should be implemented. Widgets are the building blocks of any flutter app and simply reference any element in the UI for more details on widgets please see this documentation:

<https://docs.flutter.dev/ui/widgets>

For each new page added, the page should also be added to the navigation bar. The navigation bar file can be found at the path:

<parent_dir>/baby_words_tracker/lib/pages/shared/bottom_bar.dart

CURRENT NAVIGATION STRUCTURE:

The bottom navigation bar currently has 5 items:

1. Home - HomePage (`lib/pages/home\_page.dart`)
2. Word Log - WordLogPage (`lib/pages/log\_page.dart`) - NEW
3. Add Entry - AddEntryPage (`lib/pages/add\_entry\_page.dart`) - NEW (replaces old add_text.dart)
4. Stats - StatsPage (`lib/pages/stats.dart`)
5. Settings - SettingsPage (`lib/pages/settings.dart`)

To add a new page to navigation, update `\_buildNavItems()` in `bottom\_bar.dart`.

```
return Scaffold(  
  backgroundColor: const Color(0xFF828A8F),  
  appBar: TopBar(pageName:  
    localizationService.translate("<page_name>")),
```

```
bottomNavigationBar: bottomBar(context, "<page_name>"),  
Body: <rest of the UI for the app>
```

Adding Functionality (Services)

Your UI should be separate from your logic. This project uses Provider, a dart package, to enable that separation. If a feature requires storage that is accessible between two widgets, it should be implemented as a service and accessed using Provider. A service is just a standalone class, usually extending ChangeNotifier, that performs some logic. See any of the services in the project already as reference (AuthenticationService in the auth folder and CurrentChildrenService in the util folder are great examples). Once your service is created, it should be initialized in main as a provider and then accessed in your widgets using a consumer widget. **Please read the Provider documentation here:**

<https://docs.flutter.dev/data-and-backend/state-mgmt/simple> (and maybe read this too: <https://pub.dev/packages/provider>) to understand Provider. It is a very important tool for bringing the app together and it makes data sharing between widgets easy. However, it does take some understanding to get used to. YouTube also has tutorials on this.

Phrase Tracking

Phrase tracking allows users to log multi-word phrases, from which individual words are automatically extracted. This feature works alongside word tracking.

Implementation:

- **Model**: `PhraseTracker` (`lib/data/models/phrase\_tracker.dart`)
- **Service**: `PhraseTrackerDataService`
(`lib/data/services/phrase\_tracker\_data\_service.dart`)
- **Database**: Phrases are stored in `/Child/{childId}/PhraseTracker/{phrase}` collection
- **Data Model**: See `docs/word\_phrase\_data\_model.md` for complete schema

Adding Phrase Functionality:

1. Phrases can be logged through the Add Entry page (`lib/pages/add\_entry\_page.dart`)
2. When a phrase is submitted, words are automatically extracted and linked
3. Phrases support the same features as words: notes, media attachments, language selection

4. The Word Log page (`lib/pages/log_page.dart`) displays both words and phrases

TODOs:

1. Phrase-level enrichment (similar to word enrichment) could be added in the future
2. Phrase statistics could be added to the stats page

Authentication

Auth is handled by the user model and authentication services. It uses roles assigned using custom claims. The roles are represented in an enum in utils and a corresponding enum in roles.js in the functions/auth in the firebase project folder. The user types are represented by an enum in utils.

Adding New Roles

To add new roles, update the enum in utils and in the javascript file. Make sure the string representations in both match or the app will not recognize the new roles.

Statistics

The current statistics available to the user on the home and statistics page are a good start, but much more rich information could be provided to the user with only the data already stored. The main limiting factor for new stats will be database calls, as efficiently querying childrens' histories has repeatedly raised concerns about the numbers of reads necessary. The issue of read count could be addressed by having live updating database-side statistics, such as words learned this week or number of each part of speech.

Adding New Graphs on the Stats Page:

In order to add a new graph that works in the existing system, the programmer must:

1. Add a new properly named value to the GraphType Enum, located in `<root_directory>\baby_words_tracker\lib\util\graph_type.dart`
 - a. Make sure to add its option name and display name, so that the graph will be labelled properly in the UI
2. If the graph has an adjustable time horizon, add it to the graphsWithLength set at the top of `<root_directory>\baby_words_tracker\lib\pages\stats.dart`
3. Create a data retrieval function to act as the future for the new graph's FutureBuilder
 - a. Ensure that this function uses the caching structure to prevent overquerying

4. Create a FutureBuilder that creates a syncfusion graph or other figure to represent that data
5. Add a call to that FutureBuilder to the graphSwitcher, associated with the proper GraphyType

TODOs:

1. Add more interesting statistics for the end user
 - a. e.g. Features relating to the frequency of the words the child is learning
 - i. i.e. Your child knows 500 of the top 3000 most common words!
2. Add database-end statistics to reduce querying
3. Add a caching system that either saves graphs for the entire session, for a set amount of time, or until the relevant information in the graph changes
 - a. The current caching implementation only saves data for the current instance of the stats page – extremely naive
 - b. Large potential for query saving here, too.

Cloud Functions

Cloud functions and how to update them are explained in detail in the Feature Documentation doc. Please reference it for any information on updating functions or the current functionality they have.

JavaScript Functions:

Located in `firebase-project/functions/` - see Feature Documentation for details.

Python Functions:

Located in `firebase-project/functions\_python/` - handles word enrichment using spaCy and WordNet.

- **Main file**: `firebase-project/functions\_python/main.py`
- **Setup**: See `firebase-project/functions\_python/NLTK\_SETUP.md`
- **Tests**: `firebase-project/functions\_python/tests/test\_process\_word.py`
- **Deployment**: Python functions are deployed separately from JavaScript functions

TODOs:

1. Add phrase-level enrichment to Python functions
2. Add support for more languages in Python functions
3. Improve error handling and retry logic

Data

Database Structure:

The firestore database structure is not enforced on the firebase side because of the NoSQL nature. This means that all changes to the structure need to be done in the database models and these changes will be reflected in the database for all new documents. However, this does not retroactively fix documents that are in the database already. Unfortunately, you will either need to manually update the existing documents or write a script to update the documents with the new structure. When updating using a script, the set repository function should be used since that merges with current data and does not override like the update function. Note, the update option is only available for documents that allow updates from users. You can see which documents allow for updates in the rules section of the firebase console. For more on firestore rules please see the following documentation:

<https://firebase.google.com/docs/firestore/security/get-started>

Database Repository and Services:

The database repository and services should be updated when you need to implement a new CRUD functionality. The repository should implement any new direct calls to the database. Documentation on the firestore API that is used throughout can be found here:

<https://firebase.google.com/docs/firestore/>

Services specifically implement the business logic for each of the models. These functions should make calls to the `firestore_repository` to implement CRUD functionality. When writing these functions beware of the number of reads and writes that will be called because once 50k reads are done in a day the cost increases substantially. For reads that need all of the docs from a specific collection, batch calls should be used. These are implemented in the functions that start with `get_multiple`.

TODOs:

- 1.) Currently the researcher home page is very read heavily and will easily hit the max number of reads with several calls. The fix to this is to use paging. However, due to time constraints, this has yet to be implemented.

- 2.) There are currently no processes for deleting parents, children, or researchers from the database. Deleting children is a feature that parents will likely want. However, consideration needs to be taken for what data should be deleted and which should be kept. For research purposes, it may be advantageous to keep child data in many cases.

Word Enrichment / WordBank

The app includes a word enrichment system that uses Python Cloud Functions to analyze words using spaCy and WordNet.

Implementation:

- **Python Functions**: Located in ``firebase-project/functions_python/``
- **Setup**: See ``firebase-project/functions_python/NLTK_SETUP.md`` for setup instructions
- **Service**: ``WordDataService`` (``lib/data/services/word_data_service.dart``)
- **Data Model**: See ``docs/word_phrase_data_model.md`` for complete schema

How It Works:

1. When a word is logged, it's added to the global ``/Word/{wordId}`` collection
2. Words are queued for processing using ``queueWordForProcessing()``
3. Python Cloud Function (``process_word``) processes words using spaCy and WordNet
4. Results include: lemma, part-of-speech, semantic categories
5. Enriched data is stored back in the Word document

Adding Word Enrichment Features:

1. To modify enrichment logic, edit ``firebase-project/functions_python/main.py``
2. To add new languages, ensure spaCy model is available and update language handling
3. Tests are located in ``firebase-project/functions_python/tests/``

TODOs:

1. Add phrase-level enrichment (similar to word enrichment)
2. Add more languages beyond English and Spanish
3. Improve error handling and retry logic for failed enrichments

Localization:

The main things that can be changed in localization are adding new phrases and new languages. New phrases can be added by adding a new key-value pair to the `all_localizations` file in the `l10n` folder (that is the standard abbreviation for localization). The key should be the same for all languages and then the value should reflect the text that should be displayed for each language. If a key is passed to the translation function that does not exist within the `all_localizations` file, the plain text of the key will be displayed instead of the correct translation. Adding new languages should also be straightforward. Simply add another language code to the `all_localizations` file and add the correct translations for that language. However, adding the setting to switch to a new language could provide difficulty, but the code for adding a list selection widget can be seen in the “Add Child” feature and should provide some insight on how to move forward.

TODOs:

1. A low priority issue is the formatting of the `all_localizations` file. This file is essentially one long list of every piece of text in the app at the moment. It might be advantageous to switch to having a different structure, such as a separate localization map for each page.

Media (Videos and Images)

The media upload, download, and display features support both videos and images. The functions used for media upload and download can be found in the `video` folder.

Implementation:

- **Service**: `MediaStorageService` (`lib/video/media_storage_service.dart`) - replaces old `VideoStorageService`
- **Model**: `LocalMediaEntry` (`lib/video/local_media_entry.dart`) - supports both videos and images
- **Display Page**: `DisplayMediaPage` (`lib/pages/display_media_page.dart`) - renamed from `display_video_page.dart`
- **Upload**: Media upload is implemented on the Add Entry page (`lib/pages/add_entry_page.dart`)

How It Works:

1. Users can select videos or images through the `file_picker` library
2. Media files are stored locally in the app's documents directory
3. A manifest file tracks all media entries per parent
4. Media is associated with words/phrases via the `videoid` field
5. The Display Media page shows all media for the selected child in a dropdown
6. Videos are played using Chewie video player
7. Images are displayed directly

Supported Formats:

- **Videos**: `.mp4`, `.mov`, `.m4v`
- **Images**: `.png`, `.jpg`, `.jpeg`, `.gif`, `.bmp`, `.webp`, `.heic`, `.heif`